MATLAB

3. Riadiace štruktúry, funkcie a reťazce



Čo nás dnes čaká

1. **Riadiace štruktúry** — ako program rozhodne a ako niečo zopakuje
   - vetvenie `if`, `switch`; cykly `for`, `while`; `break` a `continue`
2. **Funkcie** — ako si vytvoriť vlastný príkaz
   - vstupy a výstupy, rozsah platnosti premenných, anonymné funkcie
3. **Reťazce** — podrobná práca s textom
   - `char` vs `string`, porovnávanie, hľadanie, rozdeľovanie, formátovanie

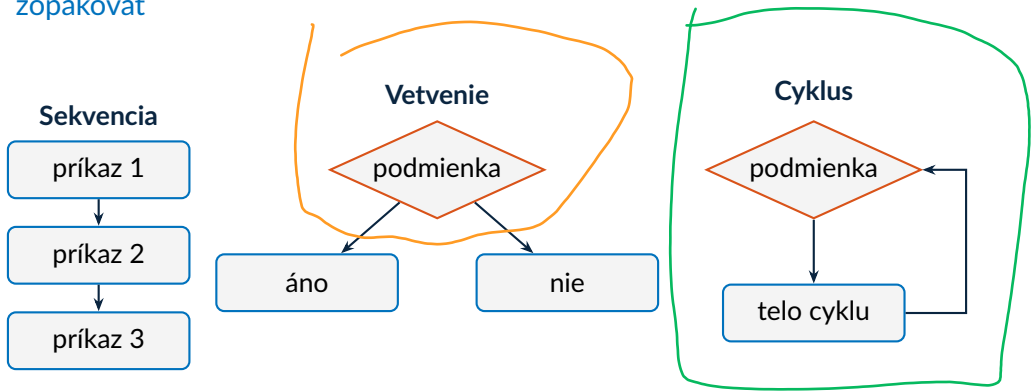
Doteraz sme počítali s číslami a maticami. Dnes sa naučíme **programovať** — teda písať kód, ktorý sa vie rozhodovať a opakovať.

1. Riadiace štruktúry

Ako beží program



- Bez riadiacich štruktúr sa príkazy vykonávajú **zhora nadol**, jeden za druhým
- **Riadiace štruktúry** tento tok menia — umožnia sa rozhodnúť alebo niečo zopakovať



Podmienka `if` — teória

- Vykoná blok príkazov **len vtedy**, ak je podmienka pravdivá
- Podmienka je **logický výraz** — to, čo sme sa učili pri relačných operátoroch
- Blok sa vždy musí ukončiť slovom `end` — MATLAB nepozná odsadenie ako Python
- Odsadzujeme kvôli čitateľnosti; zarovnať kód vieš skratkou `Ctrl+I`

```
if podmienka
    prikazy
end
```

```
teplota = 25;
if teplota > 20
    disp('Je teplo')
end
```

```
Je teplo
```

if – elseif – else

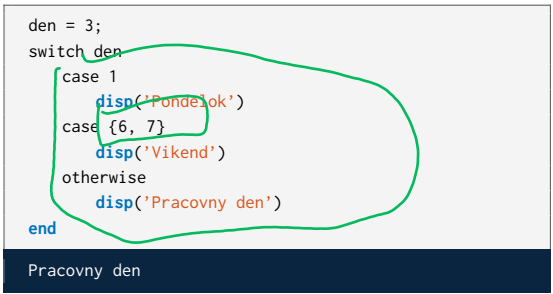
```
body = 78;
if body >= 90
    znamka = 1;
elseif body >= 75
    znamka = 2;
elseif body >= 60
    znamka = 3;
else
    znamka = 4;
end
znamka
```

JMP ADDR 1
JMP ADDR 2

```
znamka =
     2
```

- Podmienky sa testujú **zhora nadol**
- Vykoná sa len prvá splnená vetva, zvyšok sa preskočí
- `else` je nepovinné — zachytí „všetky ostatné“ prípady
- Preto stačí písať `body >= 75`, nie `body >= 75 && body < 90`

Prepínač `switch`



- Hodí sa, keď porovnávame **jednu premennú** s viacerými konkrétnymi hodnotami
- Viac hodnôt naraz zapíšeme do **zložených zátvoriek**: `case {6, 7}`
- `otherwise` je obdoba `else`
- Funguje aj s textom: `case 'ano'`
- Na rozdiel od C tu netreba `break` — vykoná sa len jedna vetva

Cyklus `for` — teória

- Použijeme, keď **vopred vieme**, koľkokrát sa má niečo zopakovať
- Riadiaca premenná postupne preberá hodnoty zo zadaného vektora

```
for i = 1:5
    prikazy
end
```

Premenná `i` nadobudne postupne hodnoty `1, 2, 3, 4, 5`.

```
sucet = 0;
for i = 1:5
    sucet = sucet + i;
end
sucet
```

```
sucet =
    15
```

Ako cyklus prebieha krok za krokom

Iterácia	`i`	`sucet` pred	výpočet	`sucet` po
1	1	0	$0 + 1$	1
2	2	1	$1 + 2$	3
3	3	3	$3 + 3$	6
4	4	6	$6 + 4$	10
5	5	10	$10 + 5$	**15**

- Premenná `sucet` sa musí **pred cyklom** nastaviť na 0 — inak by neexistovala
- Takémuto priebežnému spočítavaniu sa hovorí **akumulátor**
- Keď si nie ste istí, čo cyklus robí, vypíšte si medzikroky cez `disp` alebo `fprintf`

`for` cez prvky poľa a vnorené cykly

Cyklus nemusí ísť len `1:n` — môže prechádzať **priamo hodnoty** vektora:

```
ceny = [12 7 25 4];
for c = ceny
    fprintf('cena: %d\n', c);
end
```

```
cena: 12
cena: 7
cena: 25
cena: 4
```

Vnorený cyklus — cyklus v cykle, napr. pre riadky a stĺpce matice:

```
for i = 1:3
    for j = 1:2
        fprintf('i=%d j=%d\n', i, j);
    end
end
```

```
i=1 j=1
i=1 j=2
i=2 j=1
i=2 j=2
i=3 j=1
i=3 j=2
```

Cyklus `while`

```
x = 1;
pocet = 0;
while x < 1000
    x = x * 2;
    pocet = pocet + 1;
end
x
pocet
```

```
x =
        1024
pocet =
    10
```

- Opakuje, **kým platí podmienka** — nevieme vopred koľkokrát
- Podmienka sa testuje **pred** každým prechodom
- V tele cyklu sa musí meniť to, čo je v podmienke! Inak vznikne **nekonečný cyklus**
- Zaseknutý program zastavíš klávesami `Ctrl+C`

for = poznám počet opakovaní
while = opakujem, kým niečo platí

break a continue

`break` — **ukončí** cyklus predčasne:

```
cisla = [4 9 14 21 30];
for k = 1:length(cisla)
    if mod(cisla(k),7) == 0
        fprintf('prve: %d\n', cisla(k));
        break
    end
end
```

```
prve: 14
```

Našli sme, čo sme hľadali — nemá zmysel pokračovať.

`continue` — **preskočí** zvyšok iterácie:

```
for k = [3 -1 5 -8 2]
    if k < 0
        continue
    end
    fprintf('kladne: %d\n', k);
end
```

```
kladne: 3
kladne: 5
kladne: 2
```

Záporné čísla sa preskočia, cyklus však beží ďalej.

Vektorizácia — kedy cyklus vôbec netreba

MATLAB je stavaný na prácu s celými poľami naraz. Ten istý súčet dvoma spôsobmi:

```
s = 0;
for i = 1:length(v)
    s = s + v(i);
end
```

```
s = sum(v);
```

Pre vektor s miliónom prvkov (priemer z troch meraní):

cyklus `for`	0,0084 s
funkcia `sum`	0,0004 s

- Vektorový zápis je **asi 20× rýchlejší** — a aj kratší a čitateľnejší
- Pravidlo: ak sa to dá napísať bez cyklu, napíš to bez cyklu
- Cyklus použi vtedy, keď kroky na sebe závisia alebo treba niečo vypisovať

Riadiace štruktúry — časté chyby

- Zabudnutý `end` — najčastejšia chyba začiatočníkov; každý `if`, `for`, `while` aj `function` ho potrebuje
- `=` namiesto `==` v podmienke — `if x = 5` je chyba, správne je `if x == 5`
- **Nekonečný `while`** — v tele sa nemení premenná z podmienky
- **Predlžovanie poľa v cykle** je pomalé:

Pomaly — pole rastie v každom kroku:

```
for i = 1:1000
    v(i) = i^2;
end
```

Rýchlo — miesto si pripravíme vopred:

```
v = zeros(1,1000);
for i = 1:1000
    v(i) = i^2;
end
```

2. Funkcie

Prečo funkcie

- Doteraz sme písali skripty — postupnosť príkazov, ktorá sa vykoná odhora nadol
- **Funkcia** je pomenovaný kus kódu, ktorý niečo **dostane**, niečo **urobí** a niečo **vráti**
- Vlastne si tým vytvárame **vlastný príkaz** — rovnako ako `sum` alebo `det`

Čo tým získame:

- **Nemusíme opakovať kód** — napíšeme raz, voláme koľkokrát chceme
- **Prehľadnosť** — z dlhého skriptu sa stane pár zrozumiteľných volaní
- **Ľahšie hľadanie chýb** — funkciu vieme otestovať samostatne
- **Zmena na jednom mieste** sa prejaví všade, kde sa funkcia používa

Anatómia funkcie




```
function [vystup1, vystup2] = menoFunkcie(vstup1, vstup2)
```

- Kľúčové slovo `function`, potom výstupy, znak `=`, **meno** a v zátvorkách vstupy
- Ak je výstup len jeden, hranaté zátvorky netreba
- Telo sa ukončí slovom `end`
- Meno súboru sa musí zhodovať s menom funkcie — funkcia `obsahKruhu` patrí do súboru `obsahKruhu.m`

Prvá vlastná funkcia

Súbor `obsahKruhu.m`:

```
function S = obsahKruhu(r)
%OBSAHKRUHU Vypocita obsah kruhu.
    S = pi * r^2;
end
```

Riadok začínajúci `%` hneď pod hlavičkou je **nápoveda** — zobrazí sa príkazom `help`.

Volanie z príkazového riadku:

```
obsahKruhu(3)
S = obsahKruhu(2)
help obsahKruhu
```

```
ans =
   28.2743

S =
   12.5664

 OBSAHKRUHU Vypocita obsah
 kruhu.
```

Viac vstupov a viac výstupov

Súbor `statistiky.m`:

```
function [priemer, minimum, maximum] ...
        = statistiky(v)
    priemer = mean(v);
    minimum = min(v);
    maximum = max(v);
end
```

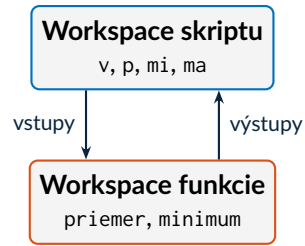
```
[p, mi, ma] = statistiky([4 8 15 16 23 42])
```

```
p =
    18
mi =
     4
ma =
    42
```

- Výstupy sa pri volaní preberajú do **hranatých zátvoriek** v rovnakom poradí
- Ak nás zaujíma len prvý výstup, stačí `p = statistiky(v)` — ostatné sa nevypočítajú zbytočne
- Znak `...` slúži na rozdelenie dlhej hlavičky na dva riadky

Rozsah platnosti premenných

- Funkcia má **vlastný pracovný priestor**
- Premenné vo funkcii nevidno zvonku a naopak
- Komunikuje sa **výhradne** cez vstupy a výstupy
- Preto sa dá funkcia bezpečne použiť kdekoľvek — neprepíše ti premenné v skripte



Anonymné funkcie

- Krátky jednoriadkový predpis, ktorý nepotrebuje vlastný súbor
- Zapisuje sa pomocou zavináča `@` a zoznamu premenných v zátvorkách

```
f = @(x) x.^2 + 1;
f(3)
f([1 2 3])

plocha = @(a,b) a*b;
plocha(3,4)
```

```
ans =
    10

ans =
     2     5    10

ans =
    12
```

Všimni si `.^` — vďaka prvkovej mocnine funguje predpis aj pre celý vektor naraz.

3. Reťazce

Dva druhy textu: `char` a `string`

`char` — pole znakov, `'jednoduché'`

```
meno = 'Jozef';
class(meno)
length(meno)
```

```
ans =
    'char'
ans =
     5
```

Správa sa ako **vektor** — každý znak je jeden prvok.

`string` — textový objekt, `"dvojité"`

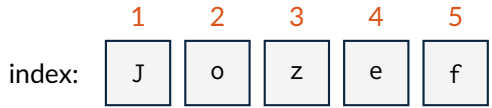
```
s = "Jozef";
class(s)
strlength(s)
```

```
ans =
    "string"
ans =
     5
```

Správa sa ako **jedna hodnota**; dĺžku zisťujeme cez `strlength`.

Novší typ `string` je pohodlnejší; `char` však stále stretneš v staršom kóde a pri práci so súbormi.

`char` je pole — dá sa indexovať




```
meno = 'Jozef';
meno(1)
meno(end)
meno(2:4)
```

```
ans =
    'J'
ans =
    'f'
ans =
    'oze'
```

Platia tu **presne tie isté pravidlá** ako pri vektoroch: indexovanie od 1, dvojbodka pre rozsah, `end` pre posledný prvok.

Spájanie textu

Pri `char` spájame ako **vektory** — hranatými zátvorkami:

```
a = 'Dobry';
b = 'den';
spolu = [a ' ' b]
```

```
spolu =
    'Dobry den'
```

Medzeru musíme pridať ručne ako `' '`.

Pri `string` stačí **plus**:

```
s1 = "Dobry";
s2 = "den";
spolus = s1 + " " + s2
```

```
spolus =
    "Dobry den"
```

Toto je hlavná výhoda typu `string`.

Porovnávanie textu — pozor!

Toto je najčastejšia chyba pri práci s textom.

```
'abc' == 'abd'
```

```
ans =
  1x3 logical array
   1   1   0
```

Operátor == porovnáva **znak po znaku** a vráti pole, nie jednu odpoveď.

```
'abc' == 'abcd'
```

```
Arrays have incompatible sizes
for this operation.
```

```
strcmp('abc','abd')
strcmp('Ahoj','ahoj')
strcmpi('Ahoj','ahoj')
```

```
ans =
  logical
   0
ans =
  logical
   0
ans =
  logical
   1
```

`strcmp` vráti **jednu** odpoveď a zvládne aj rôzne dlhé texty. `strcmpi` navyše ignoruje veľkosť písmen.

Hľadanie v texte

```
contains('programovanie','gram')
startsWith('matlab.m','matlab')
endsWith('matlab.m','.m')
strfind('abcabc','bc')
```

```
ans =
  logical
   1
ans =
  logical
   1
ans =
  logical
   1
ans =
     2     5
```

- `contains`, `startsWith`, `endsWith` odpovedajú **áno/nie**
- `strfind` vráti **pozície** všetkých výskytov — tu sa „bc“ nachádza na indexoch 2 a 5
- Ak sa nič nenájde, `strfind` vráti prázdne pole

Úpravy textu

```
upper('ahoj')
lower('AHOJ')
strtrim('  text  ')
strrep('ahoj svet', ...
      'svet','trieda')
```

```
ans =
    'AHOJ'
ans =
    'ahoj'
ans =
    'text'
ans =
    'ahoj trieda'
```

- `upper` / `lower` — veľké a malé písmená
- `strtrim` — odstráni medzery na začiatku a na konci (nie vo vnútri)
- `strrep` — nahradí **všetky** výskyty podreťazca
- Pôvodná premenná sa nemení — funkcie vracajú **nový** text

Rozdelenie a spojenie




```
casti = split("jablko,hruska,slivka", ",")
```

```
casti =
  3x1 string array
    "jablko"
    "hruska"
    "slivka"
```

```
join(["a" "b" "c"], "-")
```

```
ans =
    "a-b-c"
```

`split` a `join` sú navzájom opačné operácie. Pre typ `char` existujú obdoby `strsplit` a `strjoin`.

Takto sa napríklad rozoberie dátum: `strsplit('2024-09-23','-')` dá tri časti `'2024'`, `'09'`, `'23'`.

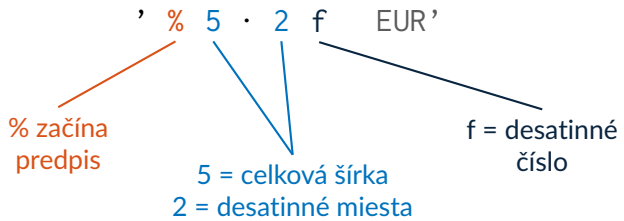
Prevod medzi číslom a textom

```
num2str(3.14159)
str2double('42')
str2double('abc')
```

```
ans =
    '3.1416'
ans =
    42
ans =
   NaN
```

- `num2str` — z čísla spraví text (napr. na výpis alebo do nadpisu grafu)
- `str2double` — z textu spraví číslo
- Ak sa prevod nepodarí, dostaneme `NaN` namiesto chyby — preto sa oplatí výsledok skontrolovať funkciou `isnan`

Formátovaný výpis — `sprintf` a `fprintf`




```
sprintf('%d kusov', 5)
sprintf('%.2f EUR', 12.3456)
sprintf('%s ma %d rokov','Jana',17)
fprintf('%d + %d = %d\n', 2,3,5);
```

```
ans =
    '5 kusov'
ans =
    '12.35 EUR'
ans =
    'Jana ma 17 rokov'
2 + 3 = 5
```

`%d` celé číslo • `%f` desatinné • `%s` text • `\n` nový riadok | **`sprintf`** text *vráti*, **`fprintf`** ho *vypíše*

Zhrnutie

- **Riadiace štruktúry**: `if`/`switch` rozhodujú, `for`/`while` opakujú — každý blok končí `end`
- **for** keď poznáme počet opakovaní, **while** keď opakujeme kým niečo platí
- Ak sa dá úloha vyriešiť **bez cyklu**, vektorový zápis je rýchlejší aj čitateľnejší
- **Funkcia** má vlastný pracovný priestor a komunikuje len cez vstupy a výstupy; meno súboru = meno funkcie
- **Text** porovnávame cez `strcmp`, nie cez `==`

Ďakujem za pozornosť

Otázky a diskusia